SEQUENCE-TO-SEQUENCE DEEP LEARNING FOR DEPENDENCY-AWARE TASK OFFLOADING IN MULTI-ACCESS EDGE COMPUTING

Mohamed Elhajji[1], Abdessamad Chaou[2], Oussama Lagnfdi[3] and Anouar Darif[4]

[1,2,3,4]LIMATI Laboratory, Department of Mathematics and Computer Science, Polydisciplinary Faculty, Sultan Moulay Slimane University, PO Box 592, Beni Mellal, 23000, Morocco

ABSTRACT

The proliferation of Internet of Things (IoT) devices and resource-intensive applications has placed growing pressure on computation, storage, and network resources. Cloud computing helps, but its centralized nature adds latency that delay-sensitive services cannot tolerate, whereas Multi-Access Edge Computing (MEC), boosted by 5G, moves computation closer to the data. Offloading remains hard, however, because applications consist of interdependent tasks while wireless conditions and edge load fluctuate continuously. This paper formulates dependency-aware task offloading as a Sequence-to-Sequence (Seq2Seq) learning problem: the application task graph is read as an input sequence, and per-task placements are produced as an output sequence. The contributions are threefold: (i) a Seq2Seq encoder-decoder model that couples a graph-aware encoder with multi-scale (global, local, and critical-path) attention; (ii) an offline curriculum-based training procedure that removes the online exploration cost of reinforcement learning; and (iii) an extensive evaluation on a discrete-event MEC simulator. The proposed model reduces application latency by up to 38% compared with Round-Robin and by about 26% compared with HEFT, with the widest margin in the 7–20 Mbps band typical of 4G networks; it scales near-linearly at about 1.7 ms per task up to 100 tasks; and it keeps the task-failure rate near 5% under overload, against 12% for Round-Robin, at a modest 18% device-energy overhead.

KEYWORDS

Internet of Things (IoT), Multi-Access Edge Computing (MEC), Quality of Service (QoS), Sequence-to-Sequence (Seq2Seq) Learning, Service Time, Task Dependency, Task Offloading

1. INTRODUCTION

The rapid expansion of computationally intensive mobile and Internet of Things (IoT) applications [1] has pushed the demand for computing resources well beyond what current smart devices can provide on their own. Emerging services such as augmented reality, intelligent transportation, real-time video analytics, and connected healthcare generate large volumes of data while demanding fast responses and reliable delivery. These requirements weigh heavily on both Quality of Service (QoS) and user experience, particularly when applications must process complex workloads under strict latency constraints. Despite steady advances in hardware, smartphones and IoT nodes remain constrained by battery capacity, storage, and processing power, making purely local execution inefficient for many demanding workloads [2].

Cloud computing has traditionally supplied the seemingly unlimited resources that such applications require. The physical distance between users and centralized data centers, however, introduces communication delays, bandwidth limitations, and congestion that undermine the

DOI: 10.5121/ijcnc.2026.18503

cloud-centric model whenever rapid decisions are needed. Multi-Access Edge Computing (MEC) was conceived to overcome these limitations by placing computational resources at the network edge, typically at cellular base stations, within a few milliseconds of the user [3], [4]. Bringing computation closer to the device reduces service latency, improves resource utilization, and raises both QoS and responsiveness.

A fundamental question in any MEC deployment is where each application task should run. Whenever a device produces a task it must decide between local execution and offloading to an edge server, and the decision is far from trivial: offloading a lightweight task over a congested uplink can be slower than computing it locally, and it costs energy in the process. The problem grows harder for realistic applications, which are rarely a single task but a graph of interdependent tasks with precedence constraints. A poor placement on the critical path stalls every task that depends on it, so decisions are coupled and the search space expands rapidly as the number of tasks increases [5].

Two families of methods dominate practice, and each falls short. Deterministic heuristics such as the Heterogeneous Earliest Finish Time (HEFT) algorithm [6] compute a complete schedule once, before execution, and cannot revise it when the channel degrades or the server becomes busy. Learning-based methods, Deep Reinforcement Learning (DRL) in particular [7], adapt online but usually treat each decision in isolation and require thousands of trial-and-error episodes during which poor exploratory choices directly harm the user. Recent advances in sequence modelling suggest a different route: Sequence-to-Sequence (Seq2Seq) architectures, designed for machine translation, learn relationships between input and output sequences and capture long-range dependencies, which is exactly the structure that couples task-offloading decisions.

In this paper, we cast dependency-aware offloading as a Seq2Seq mapping. The application task graph is read as an input sequence and the placement decisions are emitted as an output sequence, with attention supplying the coupling between decisions. Unlike conventional approaches that treat tasks independently, the model captures inter-task dependencies and the global structure of the workflow when generating decisions, and it does so without hand-crafted rules. The main contributions of this paper are summarized as follows:

1) A Sequence-to-Sequence formulation of dependency-aware offloading with a graph-aware encoder and multi-scale attention;
2) An offline curriculum-based training procedure that avoids the online exploration cost of reinforcement learning, together with a hybrid Seq2Seq-DRL extension for continuous adaptation;
3) An evaluation against HEFT and Round-Robin across bandwidth and workload sweeps and an overload stress test, with the findings distilled into deployment guidance.

The remainder of the paper is organized as follows. Section 2 reviews related work on MEC offloading and sequence models. Section 3 presents the system model and problem formulation. Section 4 details the proposed Seq2Seq strategy and its training. Section 5 reports the experimental evaluation, Section 6 discusses limitations, and Section 7 concludes.

2. BACKGROUND AND RELATED WORK

This section first reviews the fundamentals of Multi-Access Edge Computing, then surveys classical and learning-based task-offloading algorithms, introduces Sequence-to-Sequence models, and closes by identifying the research gaps addressed in this work. For a broader

treatment of the offloading landscape, the reader is referred to the recent survey by Akhlaqi and Hanapi [8].

2.1. Multi-Access Edge Computing Fundamentals

The European Telecommunications Standards Institute (ETSI) standardized MEC [3] to bring cloud capabilities to the network edge. As Taleb et al. [4] explain, MEC servers are typically deployed at base stations or aggregation points and offer single-digit millisecond latency to nearby devices. The canonical architecture comprises three layers. The device layer gathers end-user equipment, smartphones and IoT sensors, constrained in computation and energy. The MEC layer holds edge servers with substantially greater processing power. The network layer provides the wireless links, whose bandwidth varies over time. The principal advantage of MEC over conventional cloud computing is proximity: whereas datacenters may sit hundreds of kilometres away and add 50-150 ms of latency, edge servers usually reside within a few network hops and achieve sub-10 ms latency for most operations.

2.2. Task Offloading Algorithms

2.2.1. Classical and Optimization-Based Approaches

Early and still widely used solutions are heuristic or rule-based, valued for their low complexity. HEFT [6] ranks tasks by an upward measure of the work that depends on them and greedily assigns each to the processor that finishes it earliest; it is elegant and runs in $O(|V|^2)$ time, but it bases decisions solely on static characteristics and never learns from experience. The latency-classification deadline-aware scheme of Choi et al. [9] separates latency-sensitive from latency-tolerant tasks and prioritizes accordingly, and fuzzy multi-criteria schemes assign tasks by jointly weighing VM utilization, task length, network demand, and delay sensitivity [10]. A second line formulates offloading as a mathematical program: mixed-integer and greedy formulations maximize user service satisfaction in dense, delay-sensitive deployments [11], decentralized Lagrangian schemes jointly optimize latency, cost, and resource utilization for online edge scheduling [12], and joint formulations couple software caching, computation offloading, and communication-resource allocation to obtain measurable latency and energy gains [13]. Energy-aware variants further optimize power consumption and resource use in NOMA-enabled massive IoT networks [14], while game-theoretic models cast the competition for edge resources as a non-cooperative game whose equilibrium enables decentralized decisions [15]. Because the underlying assignment is NP-hard, meta-heuristics are common, with particle swarm optimization applied to industrial IoT [16] and genetic algorithms used to optimize decisions under bandwidth and compute limits [17]. Control-theoretic methods address dynamic system behaviour, for example in UAV-enabled MEC where task offloading and resource allocation are managed jointly under stability constraints [18]. Fuzzy orchestration further distributes workloads across edge nodes under uncertainty [19], and flexible fuzzy-based mobile edge orchestrators adapt offloading decisions to system context [20]. These methods are effective but rely on predefined rules or iterative search, which limits their adaptability and real-time use under fluctuating conditions.

2.2.2. Learning-Based Approaches

Recent work replaces hand-crafted rules with policies learned from data. Chen et al. [7] propose a DQN-based agent that learns offloading policies through trial-and-error interaction with the environment, and the DROO framework [21] learns binary decisions with a scalable network. These methods adapt well, yet they typically require extensive training, often thousands of episodes, can be unstable during early exploration, and, with few exceptions, treat tasks

independently rather than as a dependency graph. To the best of our knowledge, no prior work has applied Sequence-to-Sequence models to dependency-aware offloading, a notable omission given their success across sequence-prediction domains.

2.3. Sequence-to-Sequence Models

Seq2Seq models were introduced by Sutskever et al. [22] for neural machine translation. The basic architecture pairs an encoder that compresses an input sequence into a context representation with a decoder that generates the output sequence conditioned on that context. The attention mechanism of Bahdanau et al. [23] removed the fixed-length bottleneck by letting the decoder focus on different parts of the input at each step, and self-attention later dispensed with recurrence altogether [24]. Although conceived for language, the underlying idea transfers naturally to offloading: the best placement of a task depends on its characteristics, its position in the graph, and the decisions taken for related tasks, just as translating a word depends on its context.

2.4. Research Gaps

Our review reveals three gaps. First, most offloading algorithms treat decisions independently or rely on greedy heuristics, missing the opportunities that arise from considering the full task-graph structure. Second, existing learning-based methods require extensive online training, which is impractical in production systems where poor decisions during learning degrade user experience directly. Third, the literature offers little guidance on which method is best under which conditions, since results are reported as averages despite the wide variation of real network conditions. The present work addresses these gaps by using Seq2Seq to capture task dependencies, pre-training the model offline before deployment, and characterizing performance across distinct network regimes.

3. SYSTEM MODEL AND PROBLEM FORMULATION

This section introduces the network, application, computation, communication, and energy models on which the proposed strategy operates, and then states the optimization objective that guides the placement decisions.

3.1. Network and Application Model

We consider a single mobile device served by one MEC server over a wireless link, within the three-layer architecture of Figure 1. An application is modelled as a Directed Acyclic Graph (DAG) $G = (V, E)$, where $V = \{v_1, ..., v_n\}$ is the set of tasks and an edge $(i, j) \in E$ indicates that task j consumes the output of task i. Each task v_i is described by a CPU demand c_i (in megacycles), an input payload d_i^{in}, an output payload d_i^{out}, and a memory requirement m_i. A binary variable x_i encodes its placement, with $x_i = 0$ for local execution and $x_i = 1$ for offloading to the edge server.

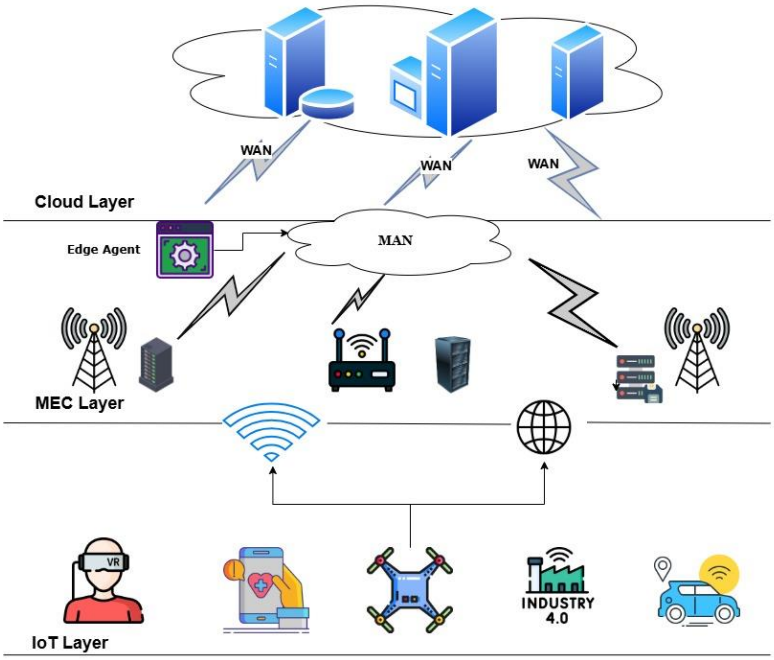


Figure 1. Three-layer Multi-Access Edge Computing architecture.

3.2. Computation and Communication Model

If task i is executed locally, its computation time is the ratio of its CPU demand to the device frequency:

$$T_i^{loc} = c_i / f_{loc} \quad (1)$$

If the task is offloaded, the device transmits the input over the wireless link, the server computes the result (possibly after queueing), and the output is returned:

$$T_i^{mec} = d_i^{in} / B + c_i / f_{mec} + T_i^{queue} + d_i^{out} / B \quad (2)$$

where B is the uplink bandwidth, f_{mec} the server frequency, and T_i^{queue} the waiting time when the server is busy. The server is assumed to have a stable power supply, so no energy constraint is imposed on it. Memory budgets must be respected at both ends, $\Sigma\, m_i \le M_{loc}$ locally and $\Sigma\, m_i \le M_{mec}$ at the edge.

3.3. Energy Model

The device spends energy on local computation and on radio transmission. When task i runs locally, the cost is $p_{loc} T_i^{loc}$; when it is offloaded, the cost is $p_{tx}\,(d_i^{in} + d_i^{out}) / B$. Only device energy enters the objective, since the server is not battery-constrained.

3.4. Optimization Objective

Task start times respect their predecessors, $start(v_i) = max$ over predecessors of $finish(v_j)$, and the application makespan is the span from the earliest start to the latest finish. The primary objective is to minimize the makespan subject to the resource constraints, while also accounting for energy and reliability. These criteria are combined into a weighted objective:

$$min\ \lambda_1 \cdot makespan + \lambda_2 \cdot \Sigma\, E_i - \lambda_3 \cdot \Sigma\, s_i \quad (3)$$

where $s_i = 1$ if task i meets its deadline and 0 otherwise, and $\lambda_1, \lambda_2, \lambda_3 \ge 0$ trade off latency, energy, and reliability. Joint placement on a DAG under these coupling and resource constraints

is a combinatorial assignment problem; since even simplified offloading formulations are NP-hard, a learned approximate policy is preferable to exact search.

4. PROPOSED SEQ2SEQ-BASED OFFLOADING STRATEGY

This section presents the proposed offloading strategy. We first explain the key insight of viewing offloading as sequence translation, and then detail the graph-aware encoder, the multi-scale attention mechanism, the decoder, the offline training procedure, and a hybrid Seq2Seq-DRL extension.

4.1. Key Insight: Tasks as Sequences

The central idea is to treat offloading as translation. The source sequence is the ordered set of tasks with their features and dependencies (analogous to source-language words and grammar); the target sequence is the placement decision for each task (analogous to target-language words); and the context is the inter-task dependency structure (analogous to semantic relationships among words). Just as translating the tenth word of a sentence depends on the preceding words and constrains those that follow, the placement of the tenth task depends on the placements of its predecessors and constrains those of its successors. This view lets us reuse the encoder-decoder machinery of neural machine translation, with attention supplying the coupling between decisions.

4.2. Graph-Aware Encoder

The encoder produces a context-rich embedding for every task by combining three signals. Feature embedding projects each task's raw attributes, namely CPU demand, data sizes, and memory, into a dense vector through a learned linear map followed by a tanh nonlinearity. Structural encoding aggregates information from a task's neighbours using the adjacency matrix

$$e_i^{struct} = tanh\ (\ \Sigma_{j \in N(i)} A_{ij} \cdot W_{struct} \cdot h_j + b_{struct}\ ) \quad (4)$$

in the spirit of graph convolution. Positional encoding adds two further signals: the task's index in a topological order and its depth in the graph, so that both sequence order and graph depth are available to the model.

4.3. Multi-Scale Attention

Rather than a single attention map, the model combines three. Global attention captures long-range dependencies across the whole graph:

$$\alpha_{i,j}^{glob} = exp(\ score(h_i, h_j)\ )\ /\ \Sigma_k\ exp(\ score(h_i, h_k)\ ) \quad (5)$$

local attention restricts the focus to graph neighbours, normalizing only over the neighbourhood N(i) and assigning zero weight elsewhere:

$$\alpha_{i,j}^{loc} = exp(\ score(h_i, h_j)\ )\ /\ \Sigma_{k \in N(i)}\ exp(\ score(h_i, h_k)\ ),\ j \in N(i) \quad (6)$$

and a critical-path bias raises the weight of tasks whose delay would delay the whole application:

$$\alpha_{i,j}^{fin} = \sigma\ (\ \alpha_{i,j}^{glob} + \alpha_{i,j}^{loc} + \beta \cdot criticality_j\ ) \quad (7)$$

where $criticality_j$ measures task j's position on the critical path and β is a learned weight.

4.4. Decoder with Auxiliary Heads

The decoder is a Long Short-Term Memory (LSTM) network that, at step t, consumes the attention context c_t and its previous state and emits a placement decision through a softmax layer:

$$d_t^{dec} = LSTM_{dec}(\ c_t,\ d_{t-1}^{dec}\ ) \quad (8)$$

$$o_t^{dec} = softmax(W_{dec} d_t^{dec} + b_{dec}) \quad (9)$$

Two auxiliary heads predict the latency and energy of the current decision through rectified-linear layers, providing additional supervision during training and exposing the predicted consequences of each placement at inference time:

$$o_t^{lat} = ReLU(W_{lat} d_t^{dec} + b_{lat}) \quad (10)$$

$$o_t^{en} = ReLU(W_{en} d_t^{dec} + b_{en}) \quad (11)$$

4.5. Offline Training and Curriculum

The model minimizes a multi-task loss that combines the placement cross-entropy with the latency and energy regression losses. Following curriculum learning [25], training proceeds from easy to hard. Stage 1 (epochs 1-10) uses simple DAGs of 10-20 tasks under stable network conditions; Stage 2 (epochs 11-25) uses medium DAGs of 20-40 tasks with moderate variability; and Stage 3 (epochs 26-80) uses complex DAGs of 40-100 tasks under highly variable conditions. This schedule improved convergence speed by roughly 30% relative to random sampling. Training offline removes the online exploration cost that burdens reinforcement learning, and the resulting policy can warm-start an online agent, as described next. The full inference procedure is summarized in Algorithm 1.

Algorithm 1. Seq2Seq-Based Dependency-Aware Task Offloading

Require: Application DAG G = (V, E); task features F; trained model θ = {Encoder, Attention, Decoder}; system state S (bandwidth B, server load, memory budgets).
Ensure: Placement x = (x_1, ..., x_n), $x_i \in$ {local, edge}; predicted latency and energy per task.

```
Sort V in topological order (v_1, ..., v_n)
for each task v_i ∈ V do
    f_i ← FeatureEmbed(F_i)
    s_i ← StructEncode(v_i, A, H)
    p_i ← PositionalEncode(i, depth(v_i))
    h_i ← f_i + s_i + p_i
end for
Build multi-scale attention {α^glob, α^loc, α^fin} over {h_i}
Initialise decoder state d_0; context c_0 ← 0
for t = 1 to n do
    c_t ← AttentionContext(d_{t−1}, {h_i}, α^fin)
    d_t ← LSTM_dec(c_t, d_{t−1})
    o_t ← softmax(W_dec d_t + b_dec)
    ℓ_t ← ReLU(W_lat d_t + b_lat); e_t ← ReLU(W_en d_t + b_en)
    x_t ← argmax o_t
    if x_t = edge and ¬Feasible(v_t, S) then
        x_t ← local
    end if
    Update system state S after placing v_t
end for
return x, {ℓ_t}, {e_t}
```

4.6. Hybrid Seq2Seq-DRL Extension

Because the model is trained on a fixed distribution, it may be suboptimal when the deployment environment differs markedly from training. We therefore augment it with a Deep Reinforcement Learning component: the Seq2Seq model supplies strong initial decisions, which a DRL agent then refines from actual deployment experience, combining the fast convergence of supervised learning with the adaptability of reinforcement learning. Offloading is formalized as a Markov decision process whose state comprises the current task together with the system

context (bandwidth, server load, remaining tasks), whose action is local or edge, and whose reward balances latency, energy, and success:

$$R(s, a, s') = -\alpha \cdot latency - \beta \cdot energy + \gamma \cdot success \quad (12)$$

where success equals 1 if the task completes on time and 0 otherwise. A full implementation and evaluation of this online stage is left to future work and is not claimed as a result here.

5. PERFORMANCE EVALUATION

This section evaluates the proposed approach. We first describe the simulation setup, and then analyse the impact of network bandwidth, scalability with task count, per-task service time, robustness under overload, and energy consumption.

5.1. Simulation Setup

The approach was evaluated on a discrete-event simulator implemented in Python that models task execution time (from CPU demand and processor frequency), wireless transmission (configurable bandwidth and latency), server queueing (tasks wait while the server is busy), and device energy (separate compute and transmit models). The simulator was validated against analytical models for simple cases and agreed within 5%. The main parameters are summarized in Table 1, and the model and training settings in Table 2. Three scenarios are considered: a bandwidth sweep from 1.2 to 65 Mbps on fixed 30-task graphs; a scalability sweep with the task count varying from 10 to 100 at a fixed 20 Mbps; and an overload stress test with 100 or more concurrent tasks. Each configuration is repeated over 100 independent trials, and means are reported with 95% confidence intervals. The proposed Seq2Seq model and its hybrid DRL variant are compared against the HEFT heuristic and Round-Robin scheduling.

Table 1. Simulation environment and workload parameters.

Parameter	Value
Device CPU frequency (f_{local})	1 GHz
MEC server CPU frequency (f_{MEC})	10 GHz
Device memory (M_{local})	2 GB
Wireless bandwidth range	1.2 – 65 Mbps
Number of tasks per application	10 – 100
CPU demand per task	50 – 500 megacycles
Input / output payload per task	0.1 – 5 MB
Independent trials per configuration	100
Reported confidence interval	95%

Table 2. Seq2Seq model and training hyperparameters.

Hyperparameter	Value
Feature-embedding dimension	128
Encoder / decoder hidden size	128 (LSTM)
Attention scales	3 (global, local, critical-path)
Optimizer	Adam
Learning rate	0.001
Batch size	32

Training samples	10,000
Training epochs (curriculum)	80
Curriculum stages	3 (10–20, 20–40, 40–100 tasks)
Loss	cross-entropy + latency/energy regression

5.2. Impact of Network Bandwidth

Figure 2 reports application latency as a function of uplink bandwidth for the four schedulers, and the curves separate into three regimes. In the low-bandwidth regime, below roughly 7 Mbps, the wireless link is the bottleneck and every method exceeds 1500 ms; the proposed model reaches about 1840 ms at 1.2 Mbps against 2300 ms for Round-Robin, already a 20% reduction obtained by offloading only compute-heavy tasks with small payloads. In the 7-20 Mbps band typical of 4G, transmission and computation costs become comparable, so the placement decision matters most: Seq2Seq latency falls from about 770 ms at 7 Mbps to about 385 ms at 20 Mbps, and its margin over the baselines is widest here. Beyond 20 Mbps, the curves flatten and converge near 220 ms as server-side processing dominates and even naive strategies perform adequately. The advantage of Seq2Seq stems from its dependency-aware, multi-criteria decisions: rather than reacting to instantaneous latency, it anticipates the downstream effect of each placement, which is precisely what matters when communication is the binding constraint.

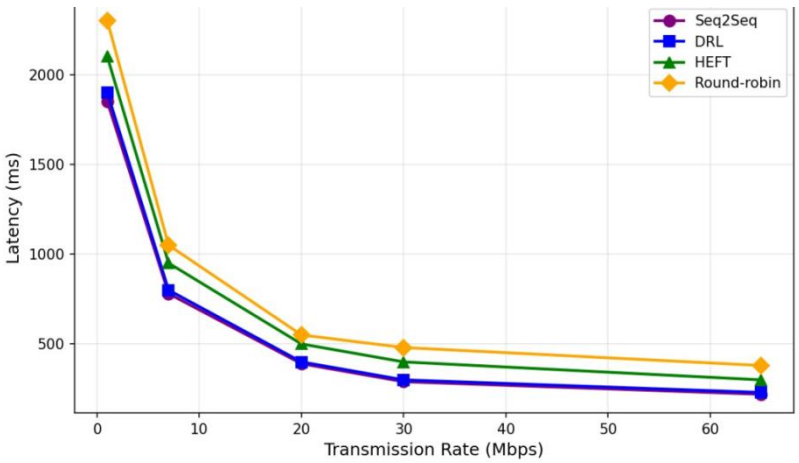


Figure 2. Application latency as a function of the network transmission rate.

5.3. Scalability with Task Count

Figure 3 examines how the schedulers scale as applications grow. All four exhibit approximately linear latency growth, which is reassuring because it points to graceful, predictable scaling rather than abrupt collapse. The proposed model has the smallest slope, close to 1.7 ms per task, so it scales best. At 100 tasks this translates into a latency of roughly 343 ms against 522 ms for Round-Robin and 450 ms for HEFT, an absolute saving of about 180 ms and 110 ms respectively, and the gap widens with task count, confirming that the dependency-aware formulation becomes more valuable as the optimization space expands. The near-linear, lowest-slope behaviour also makes the method easy to extrapolate, which is desirable for capacity planning in dense MEC-enabled IoT deployments.

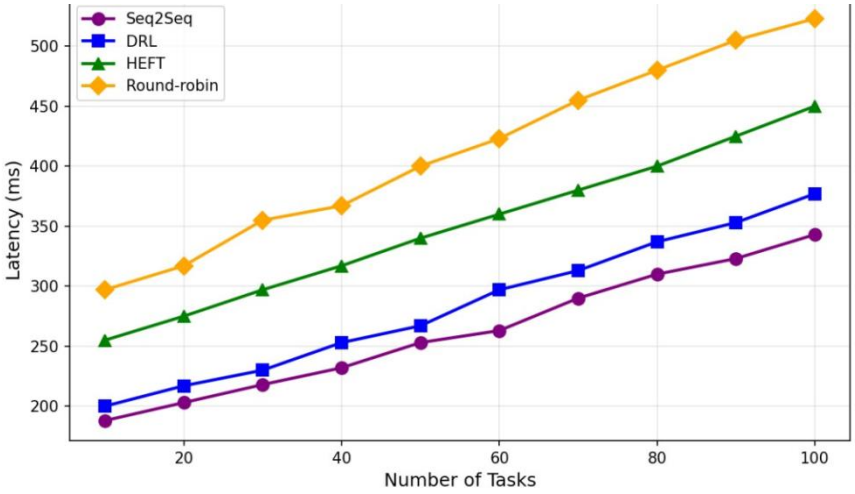


Figure 3. Total application latency as a function of the number of tasks.

5.4. Per-Task Service Time

Beyond total latency, Figure 4 reports the average service time per task, a metric that reflects per-task responsiveness directly. Service time grows slowly for every method, and the proposed model keeps the lowest values throughout, from about 16 ms at 10 tasks to 22.5 ms at 100 tasks, against 19.8 ms and 30.2 ms for Round-Robin at the two extremes, an improvement of roughly 19% to 25%, with the larger gains appearing under heavier load. Lower per-task service time matters in practice because it maps onto a more responsive user experience for interactive applications such as augmented reality and online gaming. The gain arises because the attention mechanism keeps latency-critical tasks on the fastest available resource while distributing less urgent work, whereas the fixed alternation of Round-Robin repeatedly lands critical tasks on suboptimal resources.

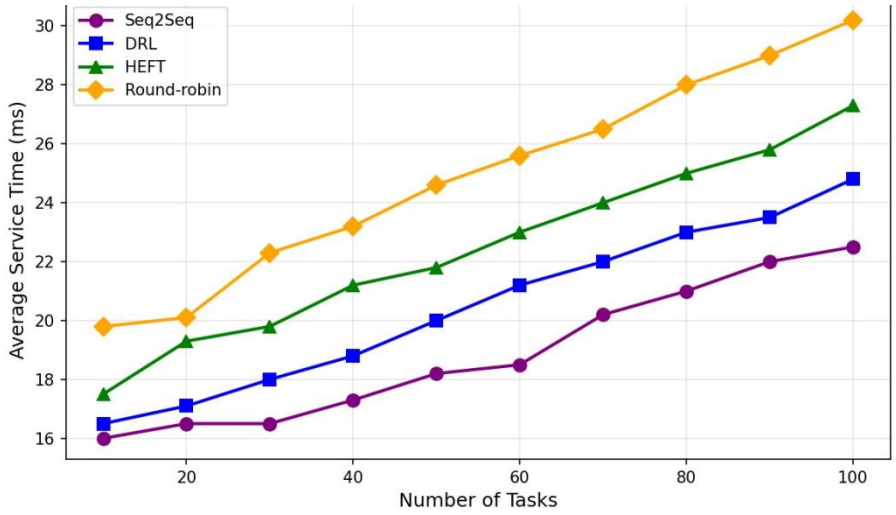


Figure 4. Average service time per task as a function of the number of tasks.

5.5. Robustness under Overload

To probe the system limits, we drove the workload toward and beyond 100 concurrent tasks. Figure 5 reports the number of failed tasks, those that miss their deadline or time out. For the

static baselines, the count climbs steeply, Round-Robin rising from about 4 failures at 80 tasks to 12 at 100, roughly a 12% failure rate, whereas the proposed model holds near 5 failures, about 5%, before any noticeable degradation. Just as important is the manner of degradation: when overloaded, Seq2Seq declines gradually rather than collapsing, because it learns to protect critical tasks when resources are scarce, a direct effect of the critical-path bias in the attention term.

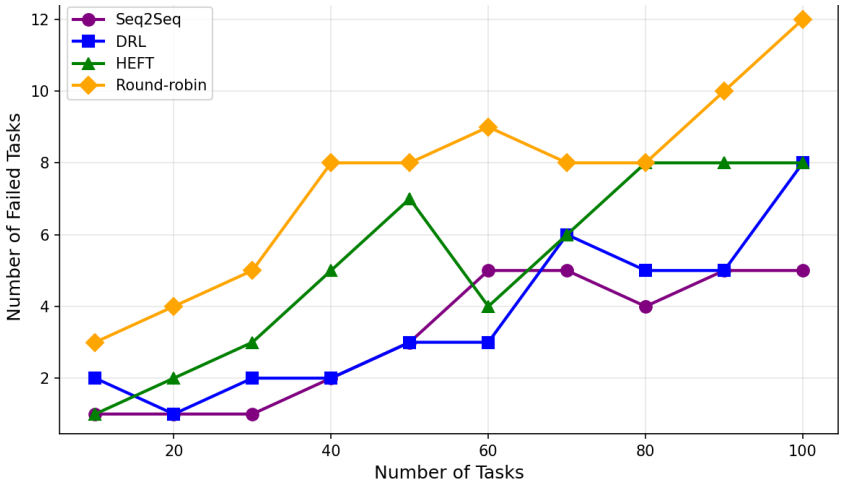


Figure 5. Number of failed tasks as a function of the total task count.

Figure 6 gives the complementary view: the number of successfully completed tasks. Across the range the two learning-based methods complete the most tasks, and at 100 tasks Seq2Seq finishes about 90 of them against 78 for Round-Robin, so the throughput advantage observed under light load is preserved and amplified under stress. Taken together, the two figures show that the approach reduces failures and increases completions at the same time, a combination that is essential for reliable operation in dense IoT environments.

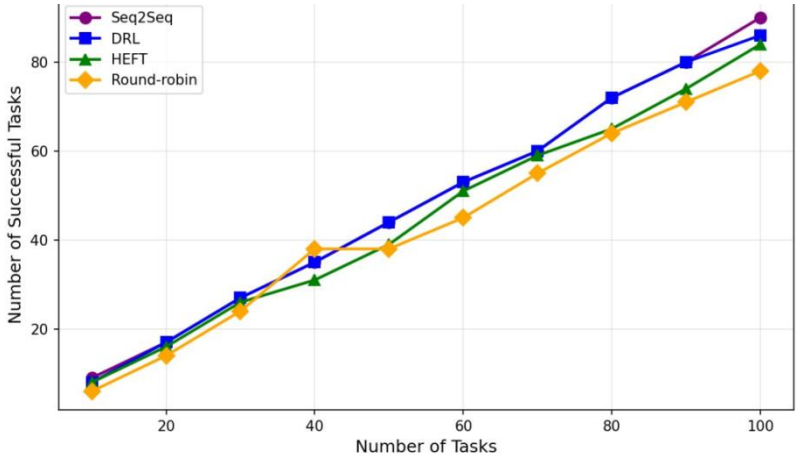


Figure 6. Number of successfully completed tasks as a function of the number of tasks.

5.6. Energy Consumption

Energy is the main cost traded against latency. Measurements indicate that the proposed model consumes about 18% more device energy than the most frugal baseline, mostly because it offloads more aggressively and adds a small on-device inference cost; in return it delivers the

latency reductions reported above. For latency-sensitive workloads such as augmented reality and online gaming this is a favourable trade-off, whereas for severely energy-constrained sensors a lighter heuristic such as HEFT may remain preferable, which underlines the value of choosing the scheduler according to the dominant deployment constraint. This observation is in line with energy-aware offloading frameworks proposed for cloud-assisted mobile environments, in which the offloading decision itself is driven by an explicit time–energy trade-off [26].

5.7. Summary of Key Findings

Table 3 synthesizes the principal outcomes. The headline result, a latency reduction of up to 38% relative to Round-Robin, concerns the dominant performance objective and is achieved across diverse scenarios. The bandwidth and scalability rows clarify where the method helps most, namely the 7-20 Mbps band that characterizes prevalent mobile networks and large task graphs where the linear, lowest-slope scaling yields predictable behaviour. The robustness and energy rows quantify the cost side of the ledger, showing that reliability is preserved up to heavy loads at a modest and bounded energy premium.

Table 3. Summary of experimental findings.

Finding	Implication
Up to 38% latency reduction vs. Round-Robin	Significant gain for latency-critical applications
About 26% latency reduction vs. HEFT	Outperforms the standard dependency-aware baseline
Largest benefit at 7–20 Mbps	Most valuable for 4G and early-5G deployments
Near-linear scaling (≈1.7 ms/task)	Predictable performance for capacity planning
Failure rate near 5% up to 100 tasks	Robust operation under realistic loads
About 18% energy overhead	Acceptable trade-off for most use cases

6. DISCUSSION AND LIMITATIONS

The principal finding is that Seq2Seq models, despite their origin in language processing, transfer remarkably well to computational offloading. The attention mechanism captures task dependencies effectively, and the model learns meaningful placement patterns from training data. The resulting latency improvement is substantial in absolute terms: for a 500 ms application, a 38% reduction brings latency to roughly 310 ms, which can change the perceived experience from sluggish to responsive.

Two limitations merit emphasis. First, training-data requirements are non-trivial: generating 10,000 training examples took roughly 40 hours of computation, and deployment in environments with markedly different network characteristics would require retraining on representative data; transfer learning may mitigate this cost but remains unexplored here. Second, interpretability is limited. Unlike HEFT, whose decisions can be traced exactly, the model behaves largely as a black box; the attention weights offer partial insight by revealing which tasks it deems important, but this is incomplete, and in safety-critical systems such as autonomous driving the opacity could be problematic. Beyond these two points, the present study considers a single MEC server, whereas practical metropolitan deployments must also select the serving cloudlet among several candidates, an edge resource-management problem

addressed in [27]; moreover, offloading traffic at the edge raises security concerns, and learning-based protection mechanisms such as actor-critic anomaly detection [28] could complement the proposed scheduler in a complete deployment.

7. CONCLUSIONS AND FUTURE WORK

This paper addressed a central question of Multi-Access Edge Computing: deciding where each task of a dependency-constrained application should run, when fixed heuristics cannot adapt to changing conditions and reinforcement learning must explore online at the user's expense. We reformulated the problem as a sequence-to-sequence translation, in which the task graph, with its features and precedence constraints, forms the input sequence and the per-task placement decisions form the output sequence. The proposed encoder-decoder architecture combines a graph-aware encoder, multi-scale attention operating at the global, local, and critical-path levels, and auxiliary heads that predict the latency and energy consequences of each decision; the model is trained entirely offline through a difficulty-based curriculum, so no exploratory mistakes ever reach real users. Extensive experiments on a discrete-event MEC simulator quantified the benefits: application latency falls by up to 38% compared with Round-Robin and by about 26% compared with the well-established HEFT algorithm, with the largest gains in the 7–20 Mbps band typical of 4G networks; the per-task cost scales near-linearly at roughly 1.7 ms per task up to 100 tasks; and under overload the task-failure rate stays near 5%, against 12% for Round-Robin, at the price of a modest 18% increase in device energy. Taken together, these results show that dependency-aware, attention-based placement is at once more effective and more robust than the standard alternatives, making the method practical for latency-sensitive mobile and IoT applications.

Will proceed along two concrete directions. First, we will fully implement and evaluate the hybrid Seq2Seq-DRL stage, so that the policy can continue to adapt online after deployment as network conditions drift. Second, we will compress the model through quantization, pruning, and knowledge distillation to reach sub-millisecond inference on resource-limited hardware. Together, these steps advance the broader goal of a self-adaptive edge-computing infrastructure.

CONFLICTS OF INTEREST

The authors declare no conflict of interest.

REFERENCES

[1] Ghasempour, A., (2019) "Internet of Things in Smart Grid: Architecture, Applications, Services, Key Technologies, and Challenges", Inventions, Vol. 4, No. 1, p22.

[2] Porambage, P., Okwuibe, J., Liyanage, M., Ylianttila, M. &Taleb, T., (2018) "Survey on Multi-Access Edge Computing for Internet of Things Realization", IEEE Communications Surveys & Tutorials, Vol. 20, No. 4, pp2961-2991.

[3] ETSI Industry Specification Group, (2019) "Multi-access Edge Computing (MEC); Framework and Reference Architecture", ETSI GS MEC 003 V2.2.1.

[4] Taleb, T., Samdanis, K., Mada, B., Flinck, H., Dutta, S. & Sabella, D., (2017) "On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration", IEEE Communications Surveys & Tutorials, Vol. 19, No. 3, pp1657-1681.

[5] Jin, X., Hua, W., Wang, Z. & Chen, Y., (2022) "A Survey of Research on Computation Offloading in Mobile Cloud Computing", Wireless Networks, Vol. 28, No. 4, pp1563-1585.

[6] Topcuoglu, H., Hariri, S. & Wu, M.-Y., (2002) "Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing", IEEE Transactions on Parallel and Distributed Systems, Vol. 13, No. 3, pp260-274.

[7] Chen, X., Zhang, H., Wu, C., Mao, S., Ji, Y. & Bennis, M., (2019) "Optimized Computation Offloading Performance in Virtual Edge Computing Systems via Deep Reinforcement Learning", IEEE Internet of Things Journal, Vol. 6, No. 3, pp4005-4018.
[8] Akhlaqi, M. Y. &Hanapi, Z. M., (2023) "Task Offloading Paradigm in Mobile Edge Computing: Current Issues, Adopted Approaches, and Future Directions", Journal of Network and Computer Applications, Vol. 212, p103568.
[9] Choi, H., Yu, H. & Lee, E., (2019) "Latency-Classification-Based Deadline-Aware Task Offloading Algorithm in Mobile Edge Computing Environments", Applied Sciences, Vol. 9, No. 21, p4696.
[10] Almutairi, J. &Aldossary, M., (2021) "A Novel Approach for IoT Tasks Offloading in Edge-Cloud Environments", Journal of Cloud Computing, Vol. 10, p28.
[11] Li, J., (2022) "Maximizing User Service Satisfaction for Delay-Sensitive IoT Applications in Edge Computing", IEEE Transactions on Parallel and Distributed Systems, Vol. 33, No. 5, pp1199-1212.
[12] Peng, Q., Wu, C., Xia, Y., Ma, Y., Wang, X. & Jiang, N., (2022) "Dosra: A Decentralized Approach to Online Edge Task Scheduling and Resource Allocation", IEEE Internet of Things Journal, Vol. 9, No. 6, pp4677-4692.
[13] Wen, W., Cui, Y., Quek, T. Q. S., Zheng, F.-C. &Jin, S., (2020) "Joint Optimal Software Caching, Computation Offloading and Communications Resource Allocation for Mobile Edge Computing", IEEE Transactions on Vehicular Technology, Vol. 69, No. 7, pp7879-7894.
[14] Liu, B., Liu, C. & Peng, M., (2021) "Resource Allocation for Energy-Efficient MEC in NOMA-Enabled Massive IoT Networks", IEEE Journal on Selected Areas in Communications, Vol. 39, No. 4, pp1015-1027.
[15] Luo, J., Qian, Q., Yin, L. &Yiao, Y., (2020) "A Game-Theoretical Approach for Task Offloading in Edge Computing", Proc. 16th Int. Conf. Mobility, Sensing and Networking (MSN), pp756-761.
[16] You, Q. & Tang, B., (2021) "Efficient Task Offloading Using Particle Swarm Optimization Algorithm in Edge Computing for Industrial Internet of Things", Journal of Cloud Computing, Vol. 10, No. 1, p41.
[17] Myyara, M., Lagnfdi, O., Darif, A. &Farchane, A., (2024) "A New Approach Based on Genetic Algorithm for Computation Offloading Optimization in Multi-Access Edge Computing Networks", IAES Int. Journal of Artificial Intelligence, Vol. 13, No. 4, pp4186-4194.
[18] Yu, Z., Gong, Y., Gong, S. & Guo, Y., (2020) "Joint Task Offloading and Resource Allocation in UAV-Enabled Mobile Edge Computing", IEEE Internet of Things Journal, Vol. 7, No. 4, pp3147-3159.
[19] Sonmez, C., Ozgovde, A. &Ersoy, C., (2019) "Fuzzy Workload Orchestration for Edge Computing", IEEE Transactions on Network and Service Management, Vol. 16, No. 2, pp769-782.
[20] Nguyen, V., Khanh, T. T., Nguyen, T. D., Hong, C. S. & Huh, E. N., (2020) "Flexible Computation Offloading in a Fuzzy-Based Mobile Edge Orchestrator for IoT Applications", Journal of Cloud Computing, Vol. 9, No. 1, pp1-18.
[21] Huang, L., Bi, S. & Zhang, Y.-J. A., (2020) "Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks", IEEE Transactions on Mobile Computing, Vol. 19, No. 11, pp2581-2593.
[22] Sutskever, I., Vinyals, O. & Le, Q. V., (2014) "Sequence to Sequence Learning with Neural Networks", Advances in Neural Information Processing Systems 27, pp3104-3112.
[23] Bahdanau, D., Cho, K. &Bengio, Y., (2015) "Neural Machine Translation by Jointly Learning to Align and Translate", Proc. Int. Conf. Learning Representations (ICLR).
[24] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. &Polosukhin, I., (2017) "Attention Is All You Need", Advances in Neural Information Processing Systems 30, pp5998-6008.
[25] Bengio, Y., Louradour, J., Collobert, R. & Weston, J., (2009) "Curriculum Learning", Proc. 26th Int. Conf. Machine Learning (ICML), pp41-48.
[26] Arya, N., Choudhary, S. & Taruna, S., (2019) "Energy Efficient Computing for Smart Phones in Cloud Assisted Environment", International Journal of Computer Networks & Communications (IJCNC), Vol. 11, No. 5, p59.
[27] Alakbarov, R. & Alakbarov, O., (2019) "Procedure of Effective Use of Cloudlets in Wireless Metropolitan Area Network Environment", International Journal of Computer Networks & Communications (IJCNC), Vol. 11, No. 1, pp93-107.
[28] Shruthi, N. & Siddesh, G. K., (2023) "Actor Critic Approach based Anomaly Detection for Edge Computing Environments", International Journal of Computer Networks & Communications (IJCNC), Vol. 15, No. 1, p53.